

OPERATOR OVERLOADING - DIFFERENT TYPES

- Overloading unary operator.
- Overloading binary operator.
- Overloading operators using a friend function.

A unary operator?

- Operators act on only one operand.
- Examples: increment (++), decrement (--), minus (-) : -33.

A binary operator?

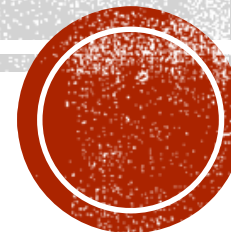
- Operators act on two operands.
- Examples: +, -, <=, >, ==,

OVERLOADING BINARY OPERATOR

- Overloading of an operator operating on two operands.
- One argument should be passed to the operator function.

PRACTICE EXERCISE

Complex Number Addition & Subtraction



```

#include<iostream>
using namespace std;

class Complex{
    int real;
    int imagi;

public:
    Complex():real(0),imagi(0){

        cout<<"Inside the constructor"<<endl;}

    Complex(int x, int y):real(x),imagi(y){

        Complex(Complex& c){
            cout<<"User Defined copy constructor"<<endl;
            real=c.real;
            imagi=c.imagi;
        }

        Complex operator +(Complex& c){
            Complex tempc;
            tempc.real=real+c.real;
            tempc.imagi=imagi+c.imagi;
            return (tempc);
        }
    }

```

```

        Complex operator -(Complex& c){
            Complex tempc;
            tempc.real=real-c.real;
            tempc.imagi=imagi-c.imagi;
            cout<<c.real<<" "<<c.imagi<<endl;
            return (tempc);
        }

        void display(){
            if(imagi < 0)
                cout << "Complex No: " << real << imagi << "i" << endl;
            else
                cout << "Complex No: " << real << "+" << imagi << "i" << endl; }

        ~Complex()
        {
            cout<<"Deallocate Memory"<<endl;
        }
    };

    int main(){
        Complex C1(3,4);
        C1.display();
        Complex C2(2,3);
        C2.display();
        C1=C2-C1;
        C1.display();
        Complex C3=C1;
        C3.display();
    }

```

```
Complex& operator =(const Complex &c){  
    real=c.real;  
    imagi=c.imagi;  
    cout<<"Inside the assignment operator"<<endl;  
    return *this;  
}
```

OVERLOADED ASSIGNMENT OPERATOR

```
ClassName& ClassName::operator=(const ClassName& op)
{
    //.....
    return *this;
}
```

ASSIGNMENT OPERATOR

- Compiler by default automatically defines an operator overloading for the assignment operator.
- Used to **copy values** from one object to another *already existing object*.

Example: obj1 and obj2

obj1 = obj2 (Copy all the data values of obj2 into obj1)

- Only shallow copy (makes a bitwise copy of the class instance) is done hence may cause problem while working with pointers.

ASSIGNMENT VS COPY CONSTRUCTOR

Feature	Copy Constructor	Assignment Operator (operator=)
Purpose	Creates a new object as a copy of an existing object	Copies the content of one existing object to another existing object
When called	During initialization of a new object	When assigning an already created object
Signature	ClassName(const ClassName &obj)	ClassName& operator=(const ClassName &obj)
Memory	Allocates new memory for the object	No new memory allocated; updates existing object
Example line	ClassName obj2 = obj1; or ClassName obj2(obj1);	obj2 = obj1;

ACTIVITY: WRITE THE OUTPUT

```
class Test {  
    public:  
        Test() {cout << "Constructor\n";}  
        Test(const Test& t) { cout << "Copy Constructor\n";}  
        Test& operator=(const Test& t) {  
            cout << "Assignment Operator\n";  
            return *this;  
        }  
};  
  
int main() {  
    Test a;  
    Test b = a;  
    Test c;  
    c = a;  
}
```

ACTIVITY

Which of the following statements invokes the assignment operator? Justify in one sentence

- A. `MyClass obj1(obj2);`
- B. `MyClass obj1 = obj2;`
- C. `obj1 = obj2;`
- D. `MyClass obj1;`

ACTIVITY

Which of the following statements invokes the assignment operator? Justify in one sentence

A. MyClass obj1(obj2);

B. MyClass obj1 = obj2;

C. obj1 = obj2;

D. MyClass obj1;

Answer: C

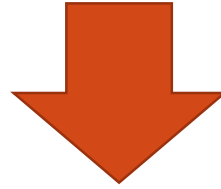
Justification:

The assignment operator is invoked when copying values into an object that has already been created.

OPERATOR FUNCTIONS: MEMBER/NON-MEMBER

- Operator functions can be either defined as members or non-members of the class
- Non-member operator functions
 - Should be defined as friend functions
 - Need access to the class's private data members.
- The way an operator function is written differs depending on whether it is a member function or a friend function.

A fundamental concept of object-oriented programming



DATA HIDING

- **Restricts access to private members from outside the class.**
- **Protected members can only be accessed by derived classes and are inaccessible from outside.**

FRIEND FUNCTIONS

- What is a friend function?
 - A friend function is a function that is not a member of a class but is granted access to its private and protected members.
- Why use a friend function?
 - Allow an external function (or another class) to access private/protected data of a class without making the data public.
 - Useful for operator overloading and functions that work on multiple classes.
- How do you define and declare a friend function?
 - **Declare** the function **inside** the class with the friend keyword.
 - **Define** it **outside** the class, similar to a regular function.

FRIEND OPERATOR FUNCTIONS

Binary operator overloaded using a friend function:

- The **left operand is passed in as the first parameter** and the **right operand is passed in as the second parameter**.

```
friend Complex& operator+(const Complex& ob1, const Complex& ob2);
```

Unary operator overloaded using a friend function:

- the operand is passed to the function as a parameter.

```
friend Counter& operator++(Counter& ob);
```

It is common for an overloaded operator function's return type to be the same as the class it operates on, so that several operations may be chained together.

An operator function that changes the value of its invoking operand normally returns a reference to that operand.

MEMBER OPERATOR FUNCTIONS

Binary operator overloaded using a member function:

- The left operand generates the function call, and the right operand is passed to the function as a parameter.

Unary operator overloaded using a member function:

- The operand generates the function call, parameter is not required.

THE ASSIGNMENT OPERATOR

- The assignment operator must be overloaded as a member function, not a friend function

[Justify (H/W)]

- The left operand invokes the **operator=()** function and accepts a constant reference to the right operand as an argument.

THE OUTPUT OPERATOR

- The output operator **must** be overloaded as a friend function because its left operand is an output stream such as **cout**.
- The general form of the **operator<<()** function is

```
ostream& operator<<(ostream& out, const ClassName& op)
{
    // ...
}
```
- The **operator<<()** function returns a reference to an **ostream** object so that output operations may be chained together.

THE INPUT OPERATOR

- The input operator must be overloaded as a friend function because its left operand is an input stream such as **cin**.
- The general form of the **operator>>()** function is
istream& operator>>(istream& in, ClassName& op)
{
 // ...
}
- The **operator>>()** function returns a reference to an **istream** object so that input operations may be chained together.



DATA CONVERSION



DATA CONVERSION

- Assignments between the same data types are handled by the compiler.
 - Basic types / user-defined types

but....

Example:

a=b;

myObj1=myObj2;

- What happens when the variables on different sides of the = are of different types?
- Basic types – Compiler handles automatically
- Compiler automatically does not handle
 - Conversions between basic types and user-defined types, and
 - Conversions between different user-defined types.

CONVERSIONS BETWEEN BASIC TYPES

- Basic Data Types
 - Float to int
 - Float to double
 - Char to int
 - int to Char etc....
- Convert the value of floatvar (expressed in floating-point format) to an integer format and assigned to intvar.

```
intvar = floatvar;
```

DATA TYPE CONVERSION

- A type conversion occurs when the compiler changes the type of an object.
- An explicit type conversion occurs when the programmer uses a cast in the code to change the type of an object.

```
double x = double(3) / 2;    // The value of x is 1.5
```

- An implicit type conversion occurs when the compiler automatically changes the type of an object.

```
int i = 32.4;                // The value of i is 32
```

EXERCISE

```
1  #include<iostream>
2  using namespace std;
3
4  int main()
5  {
6      char a;
7      float x=100.9;
8
9      a = x;
10
11     cout<<"a: "<<a<<" "<<"x:"<<x<<" "<<(int)a<<endl;
12 }
```

a: d x:100.9 100

Process exited after 0.1547 seconds with return value 0
Press any key to continue . . .

TYPE CASTING

- Converting a variable from one data type to another.
- **Implicit conversion**
 - Do not require any operator.
 - Automatically performed when a value is copied to a compatible type.
- **Explicit conversion**
 - There are many conversions
 - Imply a different interpretation of the value, require an explicit conversion.
- Casting operators:
 - `dynamic_cast`,
 - `reinterpret_cast`,
 - `static_cast` and
 - `const_cast`.



IS IT POSSIBLE?

Conversions between basic types and user-defined types ...

Conversions between different user-defined types ...



DATA CONVERSION

Conversion of primitive data type to user-defined

```
// Time Class
class Time {
    int hour;
    int mins;

public:
    // Default Constructor
    Time() : hour(0),mins(0){ }

    // Parameterized Constructor
    Time(int t)
    {
        hour = t / 60;
        mins = t % 60;
    }

    void Display()
    {
        cout << "Time = " << hour
              << " hrs and "
              << mins << " mins\n";
    }
};
```

```
int main()
{
    // Object of Time class
    Time T1;
    int dur = 95;

    // Conversion of int type to class type
    T1 = dur;
    T1.Display();
}
```



DATA CONVERSION

Conversion of class object to primitive data type

CONVERSION OF CLASS OBJECT TO PRIMITIVE DATA TYPE

- Convert from class object and a primitive data type.
- Need to use an overloaded casting operator function (conversion function)

The syntax:

```
operator typename()
{
    // Code
}
```

Example:

The **operator double()** converts a class object to type double,

The **operator int()** converts a class type object to type int etc..

```
class Time {
    int hrs, mins;

public:
    // Constructor
    Time(int, int);

    // Casting operator
    operator int();

};

Time::Time(int a, int b)
{
    hrs = a;
    mins = b;
}
```

```
// Used for Data conversion of class to primitive
Time::operator int()
{
    cout << "Conversion of Class"
        << " Type to Primitive Type"
        << endl;

    return (hrs * 60 + mins);
}

int main()
{
    int hour, mins;
    hour = 3;
    mins = 20;

    Time t(hour, mins);

    int duration = t;

    cout << "Total Minutes are "
        << duration << endl;
}
```



DATA CONVERSION

Conversion of one class type to another class type


```
// Time Class
class Time {
    int hr, mins;
public:
    // Constructors
    Time(int h, int m) : hr(h),mins(m){ }
    Time()
    {
        cout << "\nTime's Object Created";
    }

    int Minute()
    {
        int totalmin = (hr * 60) + mins;
        return totalmin;
    }

    void show()
    {
        cout << "Hour: " << hr << endl;
        cout << "Minute : " << mins << endl;
    }
};
```

```
// Minutes Class
class Minute {
    int mins;
public:
    // Constructors
    Minute()
    {
        mins = 0;
    }
    void operator=(Time T)
    {
        mins = T.Minute();
    }

    void show()
    {
        cout << "\nTotal Minute : "
        << mins << endl;
    }
};
```

```
int main()
{
    int hour, mins;
    hour = 3, mins = 40;

    Time T1(hour,mins);
    T1.show();

    Minute M1;
    M1.show();
    M1=T1;
    M1.show();
}
```